

LYCÉE GÉNÉRAL ET TECHNOLOGIQUE BAGGIO

BTS CIEL – Option Électronique et Réseaux | Année 2025-2026

RAPPORT DE PROJET

Instrumentation d'un banc d'essai pour voitures Radio Commandées (RC)

Étudiants chargés du projet :

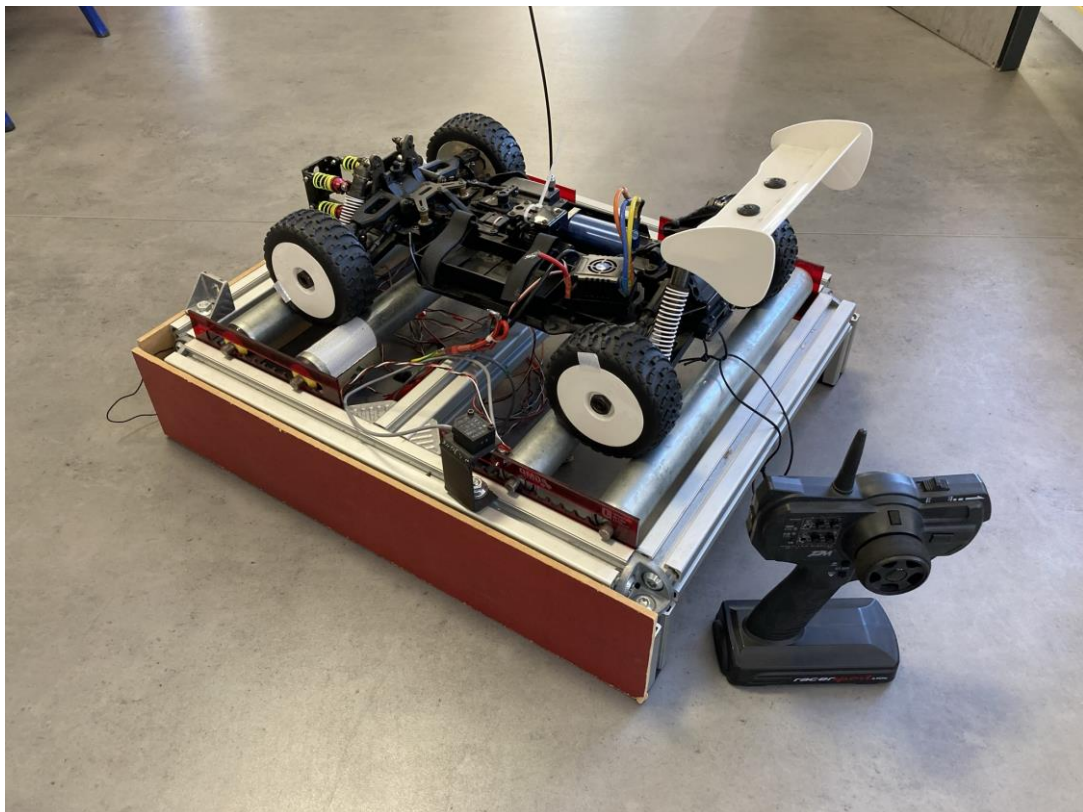
AIT AMIR Youssouf

DUCHATEAU Edouard

Professeur responsable :

Monsieur BEAU

Partenaire : FVRC – Mini RC de Marigny



Sommaire

Sommaire.....	
Remerciements.....	5
Introduction générale.....	5
Partie 1 – Contexte et identification du besoin.....	6
1.1 Contexte général du projet.....	6
1.2 Contexte applicatif : le club et son environnement.....	6
1.2.1 Zone Accueil.....	6
1.2.2 Local technique.....	6
1.2.3 Zone Piste.....	6
1.2.4 Zone Stand.....	6
1.3 Identification du besoin.....	7
1.3.1 Fonctions principales.....	7
1.3.2 Fonctions de mesure.....	7
1.3.3 Exigences non fonctionnelles.....	7
Partie 2 – Architecture du système.....	9
2.1 Vue globale de l'architecture.....	9
2.2 Flux de données.....	9
2.3 Schéma d'architecture réseau.....	9
Partie 3 – Matériel du projet.....	10
3.1 Carte Arduino UNO.....	10
3.2 Raspberry Pi 3 comme phase pilote.....	10
3.3 Raspberry Pi 5 comme phase de déploiement.....	11
3.4 Module NFC/RFID PN532.....	12
3.5 Écran Nextion NX3224T028.....	13
3.6 Capteurs de mesure.....	14
3.6.1 Capteur de vitesse LM393.....	14
3.6.2 Sonde de tension.....	14
3.6.3 Sonde de courant.....	14
Partie 4 – Mise en œuvre du module NFC/RFID.....	15
4.1 Objectif.....	15
4.2 Choix du protocole de communication.....	15
4.3 Câblage du module PN532 en I2C.....	15
4.4 Développement logiciel Arduino – Lecture NFC.....	15
Partie 5 – Serveur Raspberry Pi.....	16
5.1 Architecture du serveur.....	16
5.2 Installation et configuration d'Apache et PHP.....	16
5.3 Base de données MariaDB.....	17
5.4 Interface web PHP.....	17

5.5 Sauvegarde des données avec MySQLdump	18
Partie 6 – Réseau et VPN Tailscale	19
6.1 Problématique réseau	19
6.2 Présentation de Tailscale	19
6.3 Configuration de Tailscale sur le Raspberry Pi	19
Partie 7 – Montage final et intégration	20
7.1 Description du montage	20
7.2 Interface Nextion – conception	21
7.3 Validation du montage	21
Partie 8 – Développement logiciel	22
8.1 Structure du projet logiciel	22
8.2 Firmware Arduino	22
8.2.1 Initialisation du système	22
8.2.2 Lecture et traitement des badges NFC	22
8.2.3 Gestion de l'affichage Nextion	23
8.3 Programme Python UID_Check.py	23
8.4 Application PHP et contrôle d'accès	23
Mode API	23
Mode interface web	23
8.5 Base de données MariaDB	23
8.6 Notebook Jupyter – Analyse de données	24
Partie 9 – Phase de consolidation avant l'oral	24
9.1 Objectifs de la phase de consolidation	24
9.2 Création d'un PCB	24
9.3 Capteur LM393 – Mesure de vitesse	25
9.4 Visualisation avec Grafana	25
Partie 10 – Étape proactive : module GSM A6	25
10.1 Présentation et motivation	25
10.2 Module A6 – Caractéristiques techniques	26
10.3 Câblage et intégration	26
10.4 Fonctionnalités prévues	26
Partie 11 – Difficultés rencontrées et solutions apportées	27
11.1 Difficultés techniques	27
11.1.1 Choix du protocole pour le module PN532	27
11.1.2 Gestion des liaisons UART multiples	27
11.1.3 Configuration du Nextion Editor	27
11.1.4 Configuration du VPN Tailscale	27
11.2 Difficultés organisationnelles	27
Partie 12 – Planning et état d'avancement	29
12.1 Diagramme de Gantt – Vue d'ensemble	29
12.2 Bilan intermédiaire	29

Partie 13 – Perspectives et travaux futurs	30
13.1 Travaux restants avant l'oral.....	30
13.1.1 Finalisation du code de mesure (priorité haute)	30
13.1.2 Intégration Grafana (priorité moyenne).....	30
13.1.3 Module GSM A6 (priorité basse).....	30
13.2 Améliorations potentielles à long terme.....	30
Conclusion	31
Bibliographie et références	32
Documentation technique	32
Références logicielles	32
Normes et réglementations.....	32
Annexes	33
Annexe A – Schéma de câblage complet.....	33
Annexe B – Commandes AT du module GSM A6	33

Remerciements

Ce projet n'aurait pas été possible sans l'aide et le soutien de plusieurs personnes, que nous tenons à remercier chaleureusement.

Nous exprimons notre profonde gratitude à Monsieur BEAU, notre professeur responsable, pour son encadrement rigoureux, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses orientations techniques et pédagogiques nous ont permis de surmonter les difficultés rencontrées et de mener ce projet à bien dans les délais impartis.

Nous remercions également le club FVRC – Mini RC de Marigny, partenaire de ce projet, pour avoir mis à notre disposition le matériel nécessaire, en particulier le banc d'essai pour voitures RC, et pour nous avoir fourni le contexte applicatif concret qui a guidé l'ensemble de notre démarche.

Nous adressons aussi nos remerciements à l'équipe pédagogique du Lycée Général et Technologique Baggio, ainsi qu'à nos camarades de promotion qui ont contribué par leurs échanges et retours à l'amélioration de notre travail.

Enfin, nous remercions nos familles pour leur soutien moral et leur patience tout au long de cette année de formation.

Introduction générale

Dans le cadre de la deuxième année du BTS Cybersécurité, Informatique et réseaux, option Électronique et Réseaux (CIEL ER), au sein du Lycée Général et Technologique Baggio, nous avons été amenés à réaliser un projet technique d'envergure : l'instrumentation d'un banc d'essai pour voitures Radio Commandées (RC).

Ce projet s'inscrit dans un contexte industriel et sportif particulier : celui des compétitions de voitures RC organisées par la Fédération de Voiture Radio-Commandée (FVRC). Le club partenaire, Mini RC de Marigny, participe régulièrement à ces compétitions et éprouve le besoin de disposer d'un outil permettant de tester et d'optimiser les performances de leurs véhicules avant chaque course.

L'objectif principal de ce projet est donc de concevoir et de réaliser un système d'instrumentation complet, capable de mesurer automatiquement les performances d'une voiture RC placée sur un banc d'essai. Ce système doit être en mesure de mesurer la vitesse de rotation des roues, la tension et le courant délivrés par la batterie et le moteur, d'identifier les véhicules par lecture RFID, et d'enregistrer, d'afficher et de transmettre les résultats de manière intelligible.

Sur le plan technique, ce projet mobilise un large spectre de compétences propres au BTS CIEL ER : électronique embarquée (Arduino, capteurs), informatique industrielle (Raspberry Pi, serveur Apache, base de données MariaDB), protocoles de communication (I2C, SPI, UART), développement web (PHP, HTML), et réseautique (VPN Tailscale, Ethernet).

Ce rapport de projet intermédiaire présente l'état d'avancement du projet à la date de la revue de projet. Il décrit en détail le contexte, les objectifs, l'architecture technique, les choix technologiques réalisés, les développements effectués, ainsi que les travaux restant à accomplir avant l'oral final.

Partie commune : Partie 1 – Contexte et identification du besoin

1.1 Contexte général du projet

Le projet prend racine dans l'univers des compétitions de voitures radio-commandées, un sport technique qui exige une préparation minutieuse des véhicules. La Fédération de Voiture Radio-Commandée (FVRC) régit les règles et normes auxquelles doivent se conformer les participants lors des compétitions officielles.

Lors d'une compétition, plusieurs acteurs interviennent dans l'organisation :

- ☐ Le commissaire de course : il gère le déroulement de la course, donne les consignes de départ et d'arrêt, et veille au respect du règlement.
- ☐ L'employé (officiel de course) : il gère les inscriptions des participants, enregistre les séries et supervise les résultats.
- ☐ Le participant : il pilote sa voiture RC et consulte les résultats de ses performances.

Le système développé dans ce projet vise à automatiser le chronométrage et à fournir un banc d'essai instrumenté permettant aux équipes de valider leurs réglages avant de prendre la piste. Ce système doit respecter les normes FVRC en vigueur concernant les performances maximales autorisées pour les véhicules.

1.2 Contexte applicatif : le club et son environnement

Le club partenaire, Mini RC de Marigny (FVRC), dispose d'une infrastructure technique complète pour l'organisation de ses compétitions. Cette infrastructure se compose de plusieurs zones distinctes :

1.2.1 Zone Accueil

La zone accueil comprend un PC d'inscription pour l'enregistrement des participants, un écran d'affichage des inscriptions par série, un PC de record pour la gestion des records et des classements, un écran des scores visible des spectateurs, et une imprimante pour l'édition des documents officiels.

1.2.2 Local technique

Le local technique constitue le cœur du système informatique. Il abrite un PC serveur central qui centralise toutes les données de la compétition, ainsi qu'un commutateur réseau (switch) qui assure la connectivité Ethernet entre toutes les zones.

1.2.3 Zone Piste

La zone piste est l'espace où se déroulent les courses. Elle est équipée d'une boucle de chronométrage avec décodeur RFID relié en USB au PC Piste. Les voitures RC sont équipées d'un transpondeur RFID qui permet leur identification automatique à chaque passage sur la boucle. Un écran affiche les résultats en temps réel.

1.2.4 Zone Stand

C'est dans cette zone que s'intègre notre projet. Le stand est équipé d'un PC Banc relié en USB au banc d'essai, lui-même connecté à la voiture RC testée. Cette zone permet aux pilotes de procéder aux essais et réglages de leur véhicule avant et pendant la compétition, en consultant les mesures effectuées par notre système d'instrumentation.

1.3 Identification du besoin

L'analyse fonctionnelle du système a permis d'identifier les besoins exprimés par les différents utilisateurs. Ces besoins se traduisent par un diagramme des cas d'utilisation présentant les fonctions principales du système.

1.3.1 Fonctions principales

Le système doit répondre aux besoins suivants :

- ☐ Gérer les courses : le commissaire de course doit pouvoir démarrer et arrêter une course, ainsi que déclencher le chronométrage automatique.
- ☐ Inscrire les participants dans une série : l'employé doit pouvoir enregistrer les participants, les affecter à des séries et consulter les résultats.
- ☐ Régler la voiture RC : le participant doit pouvoir utiliser le banc d'essai pour mesurer les performances de sa voiture (tension, courant, vitesse) et effectuer les réglages nécessaires.
- ☐ Consulter les résultats à l'accueil : les résultats doivent être accessibles depuis le poste d'accueil.

1.3.2 Fonctions de mesure

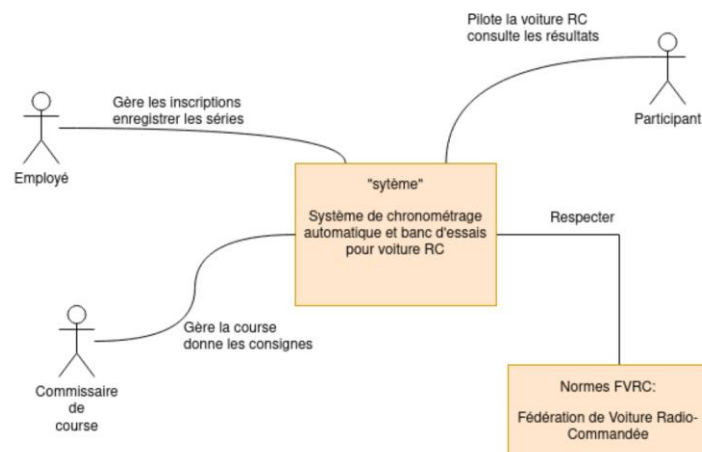
Le banc d'essai instrumenté doit être capable d'effectuer les mesures suivantes :

- ☐ Mesure de la vitesse de rotation des roues (tachymètre) : capteur LM393 avec disque codeur.
- ☐ Mesure de la tension moteur : sonde de tension (12V max).
- ☐ Mesure du courant moteur et batterie : sonde de courant (30A max).
- ☐ Lecture de l'identifiant de la voiture : lecteur RFID (protocole Mifare 13,56 MHz).

1.3.3 Exigences non fonctionnelles

Outre les fonctions de mesure, le système doit respecter certaines exigences non fonctionnelles

- ☐ Conformité aux normes FVRC concernant les performances maximales autorisées.
- ☐ Sécurisation de l'accès aux données par mot de passe.
- ☐ Mémorisation des données sur une durée d'un mois avec affichage en tableau.
- ☐ Interface web compatible smartphone pour la consultation des résultats.
- ☐ Affichage local sur écran tactile.



Partie 2 – Architecture du système

2.1 Vue globale de l'architecture

Le système d'instrumentation du banc d'essai RC repose sur une architecture à deux niveaux : un niveau acquisition/traitement local (Arduino + Nextion) et un niveau serveur/stockage/diffusion (Raspberry Pi + réseau).

Les principaux blocs fonctionnels du système sont :

- ❑ Mesures automatiques : acquisition des données de vitesse, tension, courant et couple via des capteurs connectés à une carte Arduino.
- ❑ Identification des voitures : lecture du tag RFID embarqué sur chaque voiture, via un module PN532 en protocole I2C, relié à l'Arduino.
- ❑ Serveur de données : un Raspberry Pi héberge un serveur Apache, une base de données MariaDB/MySQL, ainsi qu'une interface phpMyAdmin et Grafana pour la visualisation.
- ❑ Affichage des données : interface web (HTML/PHP) accessible depuis n'importe quel appareil connecté au réseau, ainsi qu'un affichage local sur écran tactile Nextion.
- ❑ Alimentation de la partie instrumentée : alimentation AC/DC dimensionnée pour alimenter l'ensemble du système d'instrumentation.

2.2 Flux de données

Le flux de données suit le cheminement suivant :

- ❑ Les capteurs (vitesse, tension, courant) et le module RFID transmettent leurs données à la carte Arduino via les interfaces I2C, SPI ou analogique.
- ❑ L'Arduino traite les données brutes, effectue les calculs (couple moteur, taux de charge batterie) et les transmet au Raspberry Pi via liaison série USB.
- ❑ Le Raspberry Pi, agissant comme serveur, stocke les données en base de données (MariaDB) et les rend accessibles via une interface web PHP.
- ❑ L'écran Nextion affiche en temps réel les données locales via liaison UART.
- ❑ Le VPN Tailscale permet un accès distant sécurisé au serveur depuis n'importe quel réseau.

2.3 Schéma d'architecture réseau

Le réseau du stand s'intègre dans l'architecture globale de la compétition via une connexion Ethernet au commutateur du local technique. Le Raspberry Pi (serveur Banc) est accessible depuis le PC Banc via USB et depuis n'importe quel appareil connecté au réseau Tailscale.

Composant	Rôle	Interface
Arduino UNO	Acquisition et traitement local	I2C, UART, Analogique, USB
Module PN532	Lecture RFID/NFC	I2C (adresse 0x24)
Écran Nextion	Affichage tactile local	UART (9600 bds)
Capteur LM393	Mesure vitesse (RPM)	Signal numérique PWM
Sonde tension	Mesure tension (12V max)	Analogique (ADC Arduino)

Sonde courant	Mesure courant (30A max)	Analogique (ADC Arduino)
Raspberry Pi 3B+	Serveur web/BDD	Ethernet, USB, GPIO
MariaDB	Base de données	SQL via PHP
Apache 2	Serveur web	HTTP (port 80)
Tailscale	VPN mesh	WireGuard / IP Tailscale

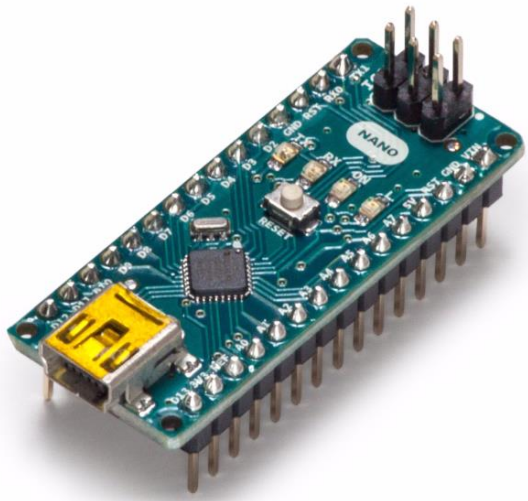
Partie 3 – Matériel du projet

3.1 Carte Arduino UNO

La carte Arduino UNO constitue le cœur du système d'acquisition local. Basée sur le microcontrôleur ATmega328P, elle offre les caractéristiques suivantes :

- ☐ Tension de fonctionnement : 5V.
- ☐ 14 broches d'entrées/sorties numériques (dont 6 PWM).
- ☐ 6 entrées analogiques (résolution 10 bits, soit 1024 valeurs).
- ☐ Fréquence d'horloge : 16 MHz.
- ☐ Interface de communication : UART, SPI, I2C.
- ☐ Connectivité USB pour programmation et communication série.

Dans notre projet, l'Arduino UNO joue plusieurs rôles simultanément : elle lit les données des capteurs (via les entrées analogiques et numériques), communique avec le module RFID PN532 (via I2C), pilote l'écran Nexion (via UART), et transmet les données traitées au Raspberry Pi (via USB/UART).



3.2 Raspberry Pi 3 comme phase pilote

Le Raspberry Pi 3 Model B+ est un nano-ordinateur monocartes basé sur un processeur ARM Cortex-A53 quad-core à 1,4 GHz. Il embarque 1 Go de RAM, un port Ethernet Gigabit, le WiFi 802.11ac et le Bluetooth 4.2. Dans notre projet, il remplit le rôle de serveur central :

- ☐ Hébergement du serveur web Apache 2.
- ☐ Gestion de la base de données MariaDB (compatibilité MySQL).
- ☐ Interface phpMyAdmin pour l'administration de la base de données.

- ❑ Visualisation des données avec Grafana.
- ❑ Connexion VPN via Tailscale pour l'accès distant.

Le Raspberry Pi est installé sous Raspberry Pi OS (basé sur Debian 12 Bookworm). Il dispose d'une adresse IP fixe sur le réseau Tailscale (100.68.68.68) permettant son accès depuis n'importe quel appareil connecté au VPN.

Grâce à cette plateforme de test, les différentes briques logicielles ont pu être validées individuellement avant leur intégration dans l'architecture finale.

Cette phase pilote a permis :

- l'installation et la configuration du système Raspberry Pi OS
- la mise en place du serveur Apache
- l'installation de PHP
- l'installation et la configuration de MariaDB
- le déploiement de phpMyAdmin
- les premiers essais de communication réseau
- la validation du VPN Tailscale.



3.3 Raspberry Pi 5 comme phase de déploiement

Après validation de l'ensemble des fonctionnalités sur le Raspberry Pi 3B+, le Raspberry Pi 5 a été retenu comme plateforme applicative finale du projet.

Ce choix s'explique par :

- une puissance de calcul supérieure ;
- une meilleure réactivité du serveur web ;
- une gestion plus performante de la base de données ;
- une meilleure évolutivité pour les futures améliorations du banc d'essai.

Le Raspberry Pi 5 héberge aujourd'hui :

- le serveur Apache ;

- l'application PHP de contrôle d'accès ;
- la base de données MariaDB ;
- les journaux d'accès RFID ;
- les futures extensions du projet.

Cette démarche a permis de séparer clairement les phases de validation technique et de mise en production.

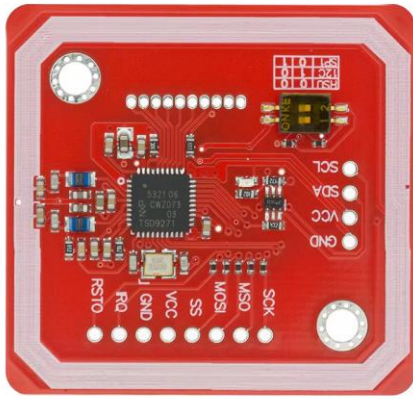


3.4 Module NFC/RFID PN532

Le module PN532 est un circuit intégré NFC/RFID développé par NXP Semiconductors. Il supporte plusieurs protocoles de communication sans contact, notamment le protocole Mifare à 13,56 MHz utilisé par les transpondeurs des voitures RC. Ses caractéristiques principales sont :

- ☐ Fréquence de fonctionnement : 13,56 MHz.
- ☐ Protocoles supportés : ISO 14443A/B, Mifare Classic, FeliCa, ISO 18092.
- ☐ Interfaces de communication disponibles : I2C, SPI, UART (HSU).
- ☐ Distance de lecture typique : jusqu'à 5 cm selon l'antenne.
- ☐ Tension d'alimentation : 3,3V ou 5V selon la version.

Dans notre projet, nous utilisons le mode I2C pour la communication entre le module PN532 et l'Arduino UNO, en raison de sa simplicité de câblage (seulement 4 fils : VCC, GND, SDA, SCL) et de la disponibilité de bibliothèques Arduino bien documentées.

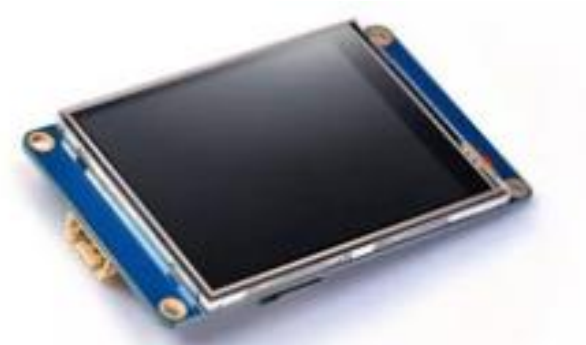


3.5 Écran Nextion NX3224T028

L'écran Nextion est un afficheur tactile couleur TFT doté d'un processeur intégré permettant de gérer son propre affichage de manière autonome. Le modèle NX3224T028 utilisé dans notre projet présente les caractéristiques suivantes :

- ☐ Taille de l'écran : 2,8 pouces.
- ☐ Résolution : 320 x 240 pixels.
- ☐ Interface de communication : UART (9600 bps par défaut).
- ☐ Mémoire Flash : 4 Mo.
- ☐ RAM : 3584 octets.
- ☐ Logiciel de conception : Nextion Editor.

L'interface graphique de l'écran Nextion est conçue à l'aide du logiciel Nextion Editor, qui permet de créer des pages d'affichage avec des composants interactifs (textes, boutons, jauges, etc.) et de définir leur comportement. Les fichiers de configuration (.HMI) sont ensuite téléversés dans la mémoire Flash de l'écran.



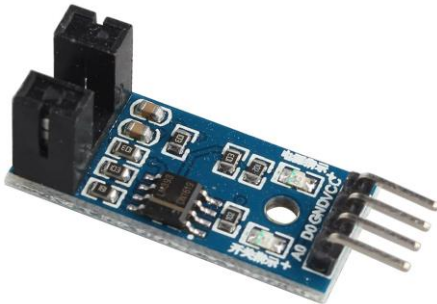
3.6 Capteurs de mesure

3.6.1 Capteur de vitesse LM393

Le capteur de vitesse utilisé est un module à fourche optique basé sur le comparateur LM393, associé à un disque codeur (encoder disk). Le principe de fonctionnement est simple : le disque

codeur est solidaire de l'axe du rouleau du banc d'essai ; à chaque rotation, les encoches du disque interrompent le faisceau lumineux infrarouge de la fourche, générant un signal numérique (PWM). La fréquence de ce signal est proportionnelle à la vitesse de rotation.

Le capteur accepte une alimentation de 5V et génère un signal logique compatible avec les entrées numériques de l'Arduino. Il peut mesurer des vitesses allant jusqu'à 30 000 tr/min, ce qui est cohérent avec les vitesses de rotation des moteurs brushless des voitures RC de compétition.



3.6.2 Sonde de tension

La sonde de tension permet de mesurer la tension aux bornes du moteur et de la batterie. Elle est basée sur un pont diviseur de tension qui ramène la tension mesurée (12V max) à une valeur compatible avec l'entrée analogique de l'Arduino (0-5V). La valeur numérisée est ensuite convertie en tension réelle par la formule : $V_{\text{mesurée}} = V_{\text{ADC}} \times (R1 + R2) / R2 \times V_{\text{ref}} / 1024$.

3.6.3 Sonde de courant

La sonde de courant permet de mesurer le courant consommé par le moteur et la batterie (30A max). Elle est basée sur le principe de l'effet Hall, qui génère une tension proportionnelle au courant qui le traverse, sans contact galvanique. Cette tension est lue par une entrée analogique de l'Arduino et convertie en ampères.

Partie Personnelle : Partie 4 – Mise en œuvre du module NFC/RFID

4.1 Objectif

Le module NFC/RFID est un élément clé du système d'instrumentation. Il permet l'identification automatique de chaque voiture RC grâce à la lecture du tag RFID (transpondeur) fixé sur le châssis du véhicule. Cette identification permet d'associer les résultats de mesure (vitesse, tension, courant) à la bonne voiture et au bon pilote dans la base de données.

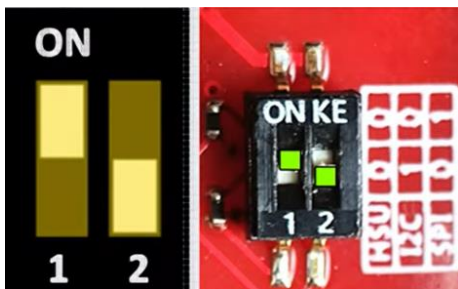
4.2 Choix du protocole de communication

Le module PN532 supporte trois protocoles de communication : I2C, SPI et UART (HSU). Le choix du protocole a une incidence sur le nombre de broches utilisées et la complexité du câblage.

Protocole	Broches utilisées	Avantages	Inconvénients
-----------	-------------------	-----------	---------------

I2C	4 (VCC, GND, SDA, SCL)	Peu de câbles, adressable	Vitesse modérée
SPI	6 (VCC, GND, MISO, MOSI, SCK, SS)	Rapide, fiable	Plus de câbles
UART (HSU)	4 (VCC, GND, TX, RX)	Simple, nombreuses bibliothèques	Partage port série Arduino

Nous avons choisi le protocole I2C car il minimise le nombre de connexions physiques nécessaires, libère les broches UART de l'Arduino pour la communication avec l'écran Nextion, et est bien pris en charge par la bibliothèque Adafruit_PN532. La configuration I2C du module est activée en positionnant les cavaliers (switches DIP) en position 1-ON, 2-OFF.



4.3 Câblage du module PN532 en I2C

Le câblage du module PN532 avec l'Arduino UNO en mode I2C est le suivant :

Broche PN532	Broche Arduino UNO
VCC (5V)	5V
GND	GND
SDA	A4 (SDA)
SCL	A5 (SCL)

4.4 Développement logiciel Arduino – Lecture NFC

Le code Arduino pour la lecture des tags NFC repose sur la bibliothèque Adafruit_PN532. Voici les étapes principales du programme :

- ❑ Initialisation : au démarrage, le programme initialise la liaison série (115200 bauds), la liaison série logicielle pour Nextion (9600 bauds), et le module NFC. Un message de confirmation est affiché sur le moniteur série et sur l'écran Nextion.
- ❑ Détection de tag : dans la boucle principale (loop()), le programme appelle `nfc.tagPresent()` pour détecter la présence d'un tag. Lorsqu'un tag est détecté, il est lu avec `nfc.read()`.
- ❑ Extraction de l'UID : l'UID (identifiant unique) du tag est extrait avec `tag.getUidString()`. Cet identifiant hexadécimal (ex : 4B C2 CD 01) est unique à chaque carte RFID.
- ❑ Affichage et transmission : l'UID est affiché sur le moniteur série, sur l'écran Nextion, et transmis au Raspberry Pi pour enregistrement en base de données.

Les résultats obtenus montrent que le système est capable de lire de manière fiable les UUIDs des tags Mifare Classic 1K utilisés comme transpondeurs. Le temps de réponse mesuré est inférieur à 200 ms, ce qui est largement suffisant pour une identification en temps quasi-réel.

Partie 5 – Serveur Raspberry Pi

5.1 Architecture du serveur

Le Raspberry Pi constitue le serveur central du système d'instrumentation. Son rôle est triple : stocker les données de mesure en base de données, les rendre accessibles via une interface web, et permettre un accès distant sécurisé via VPN. Pour ce faire, nous avons installé et configuré les logiciels suivants :

- ☐ Apache 2 : serveur web qui héberge les pages HTML/PHP de l'interface utilisateur.
- ☐ PHP 8.4.16 : langage de programmation côté serveur pour la génération dynamique des pages web et l'interaction avec la base de données.
- ☐ MariaDB 11.8.6 : système de gestion de base de données relationnelle, compatible MySQL, qui stocke les données des utilisateurs et des performances.
- ☐ phpMyAdmin 5.2.2 : interface web d'administration de la base de données.
- ☐ Grafana : outil de visualisation de données permettant de créer des tableaux de bord graphiques à partir des données MySQL.
- ☐ Tailscale : client VPN mesh permettant l'accès distant sécurisé.

5.2 Installation et configuration d'Apache et PHP

L'installation du serveur web Apache 2 et du module PHP se fait via le gestionnaire de paquets apt de Raspberry Pi OS :

```
sudo apt install apache2
```

```
sudo apt install php libapache2-mod-php
```

Après installation, le serveur Apache est opérationnel et accessible sur le port 80. Un fichier de test PHP (test.php) a été créé pour vérifier le bon fonctionnement de l'interpréteur PHP :

La page phpinfo() affiche les informations de configuration du serveur, confirmant que PHP est correctement installé et configuré avec les extensions nécessaires (mysqli, curl, mbstring, sodium).

5.3 Base de données MariaDB

La base de données MariaDB centralise l'ensemble des informations utilisées par le projet.

La table utilisateurs contient les informations relatives aux personnes autorisées à accéder au banc d'essai. Chaque utilisateur est associé à un UID de carte NFC, un nom et un prénom. Lorsqu'un badge est présenté devant le lecteur, cette table est interrogée afin de vérifier les droits d'accès.

La table performances est destinée à l'enregistrement des données issues du banc d'essai. Elle contient notamment les informations relatives aux mesures effectuées sur les véhicules radiocommandés telles que la distance parcourue, la vitesse ou encore l'état du système.

Une troisième table, nommée `logs_acces`, a été mise en place afin d'assurer la traçabilité du système de contrôle d'accès. Chaque lecture NFC y est enregistrée avec :

- l'UID détecté ;
- le nom de l'utilisateur ;
- le prénom ;
- l'état de l'accès ;
- la date et l'heure du contrôle.

Cette journalisation permet de conserver un historique complet des accès autorisés et refusés.

5.4 Interface web PHP

L'interface web constitue le point central de consultation du système de contrôle d'accès. Elle est développée en PHP et hébergée sur le Raspberry Pi 5.

Le fichier principal `check_uid.php` assure deux fonctions distinctes.

La première fonction correspond à une API utilisée par le programme Python `UID_Check.py`. Lorsqu'un UID est reçu, le script PHP interroge la base MariaDB afin de vérifier si cet identifiant correspond à un utilisateur autorisé. Si l'utilisateur existe dans la base de données, le script renvoie la chaîne « OK ». Dans le cas contraire, il renvoie « KO ».

La seconde fonction correspond à l'interface utilisateur accessible depuis un navigateur web. Cette interface affiche le résultat du dernier contrôle d'accès réalisé. Afin d'améliorer l'esthétique générale, l'arrière-plan utilise une photographie du banc d'essai.

La page se rafraîchit automatiquement toutes les cinq secondes grâce à une balise HTML spécifique. Cette solution permet d'obtenir un affichage quasi temps réel sans avoir recours à des technologies plus complexes.

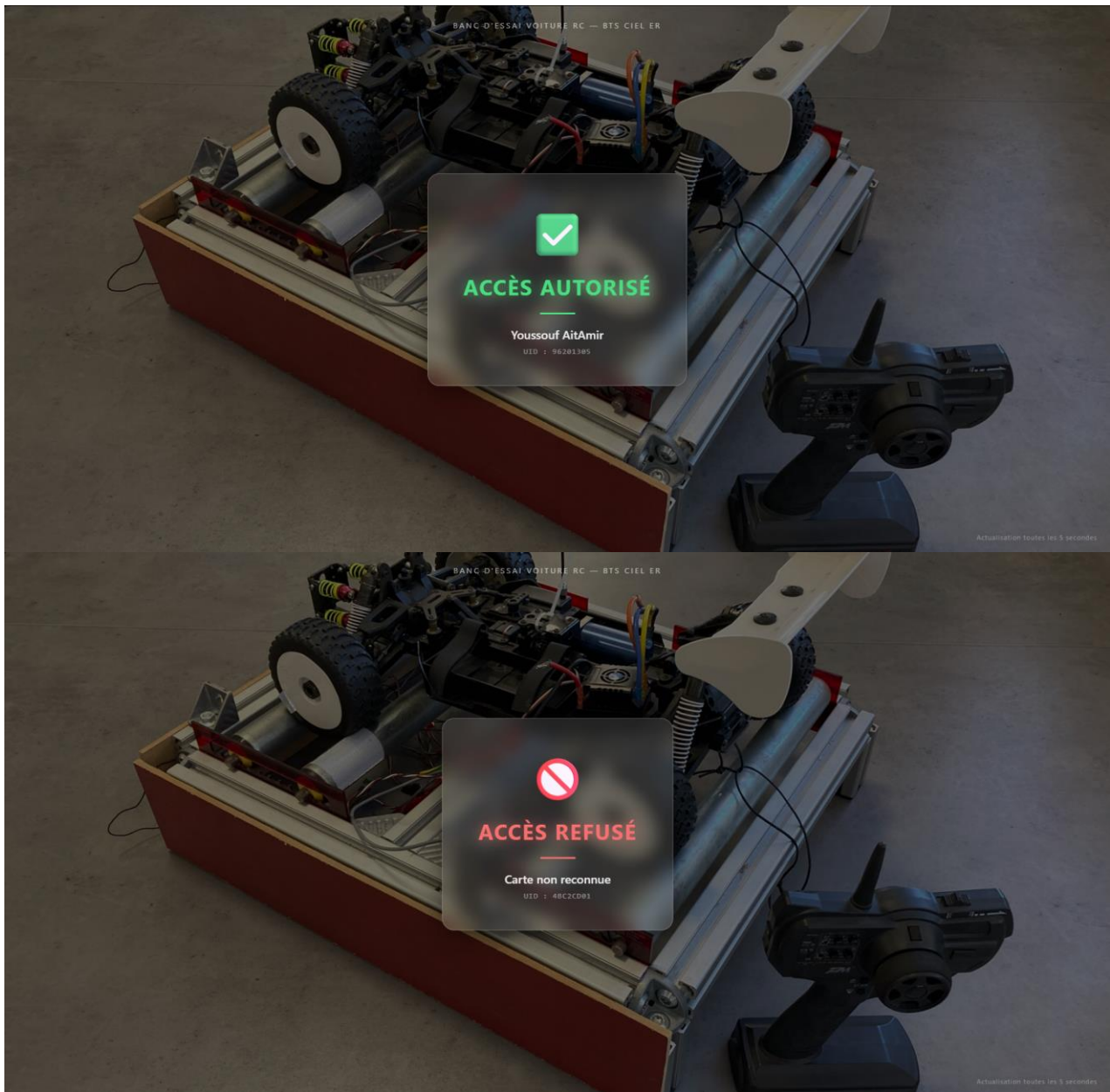
Lorsque l'accès est autorisé, l'interface affiche :

- un pictogramme vert ;
- le message « Accès autorisé » ;
- le nom et le prénom de l'utilisateur ;
- l'UID de la carte utilisée.

Lorsqu'une carte inconnue est détectée, l'interface affiche :

- un pictogramme rouge ;
- le message « Accès refusé » ;
- l'UID détecté ;
- l'indication « Carte non reconnue ».

Cette interface fournit ainsi une visualisation claire et immédiate du résultat de l'authentification.



5.5 Sauvegarde des données avec MySQLdump

Pour garantir la pérennité des données, un système de sauvegarde automatique a été mis en place avec l'utilitaire mysqldump. Une tâche planifiée (cron job) exécute quotidiennement une sauvegarde de la base de données :

```
mysqldump -u root -p banc_essai > /backup/backup_$(date +%Y%m%d).sql
```

Les sauvegardes sont conservées pendant 30 jours, conformément à l'exigence de mémorisation des données sur un mois.

Partie 6 – Réseau et VPN Tailscale

6.1 Problématique réseau

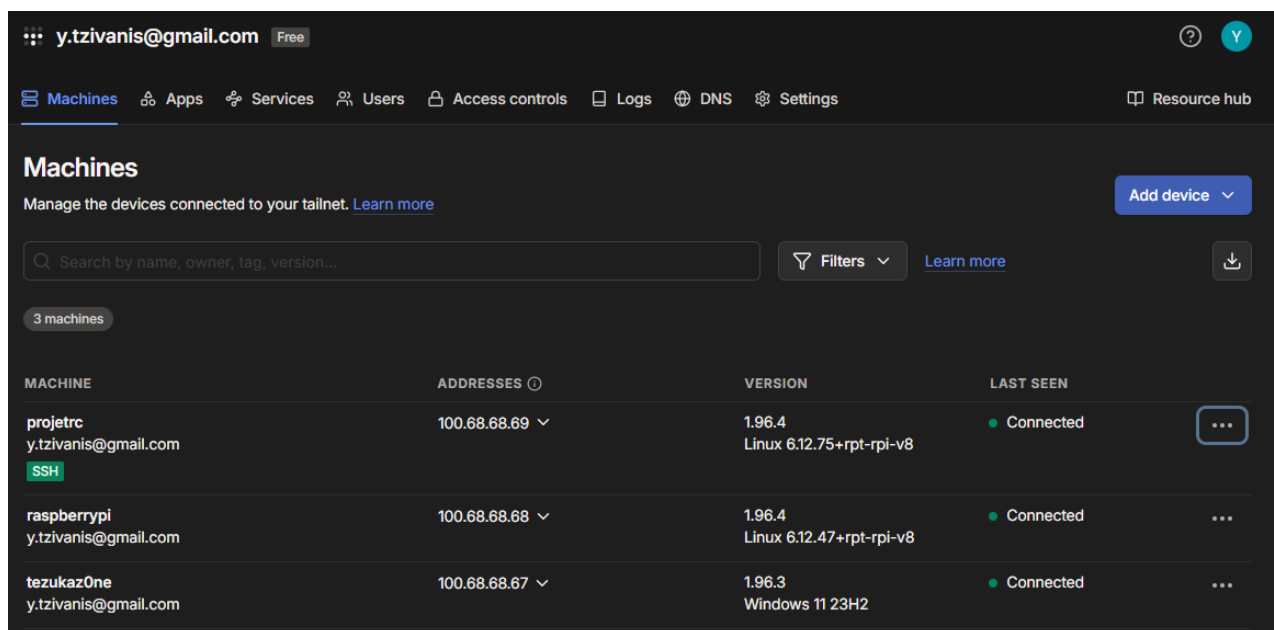
L'une des exigences du projet est de permettre l'accès aux données du banc d'essai depuis des appareils situés sur des réseaux différents (réseau local de la salle de compétition, réseau mobile, réseau du lycée). Cette exigence pose un problème de routage : le Raspberry Pi est sur un réseau local privé, et il n'est pas possible de le rendre directement accessible depuis l'extérieur sans configuration complexe du routeur.

Pour résoudre ce problème, nous avons choisi d'utiliser Tailscale, un VPN mesh basé sur le protocole WireGuard. Tailscale crée un réseau privé virtuel (tailnet) auquel peuvent se connecter tous les appareils autorisés, quelle que soit leur localisation géographique.

6.2 Présentation de Tailscale

Tailscale est une solution VPN moderne qui se distingue des VPN traditionnels par son architecture mesh : chaque appareil connecté peut communiquer directement avec les autres, sans passer par un serveur central (sauf en cas de nécessité de relais). Ses avantages pour notre projet sont :

- ☐ Installation simple : une seule commande suffit pour installer et configurer le client Tailscale sur Raspberry Pi OS.
- ☐ Authentification sécurisée : l'authentification se fait via un compte Google ou GitHub, avec génération d'un token d'authentification.
- ☐ Attribution d'adresses IP stables : chaque appareil reçoit une adresse IP fixe dans l'espace d'adressage Tailscale (100.x.x.x), indépendante de son adresse IP réelle.
- ☐ Chiffrement fort : toutes les communications sont chiffrées de bout en bout avec WireGuard.



6.3 Configuration de Tailscale sur le Raspberry Pi

L'installation de Tailscale sur le Raspberry Pi se fait en trois étapes :

- ☐ Installation du paquet Tailscale via le gestionnaire de paquets.
- ☐ Authentification sur le serveur Tailscale avec la commande 'sudo tailscale up', qui génère une URL d'authentification.
- ☐ Récupération de l'adresse IP Tailscale avec 'tailscale ip'.

Trois machines sont actuellement enregistrées dans le tailnet du projet :

Machine	Adresse IP Tailscale	OS / Version	Rôle
raspberrypi3	100.68.68.69	Linux / Tailscale 1.96.4	Phase pilote
raspberrypi5	100.68.68.68	Linux / Tailscale 1.96.4	Phase dépoloiment
tezukaz0ne	100.68.68.67	Windows 11 23H2 / 1.96.3	PC développement

Grâce à Tailscale, le Raspberry Pi est accessible depuis le PC de développement via son adresse IP Tailscale (100.68.68.68), avec un ping de 55 ms. Le serveur web Apache est accessible sur le port 80 de cette adresse.

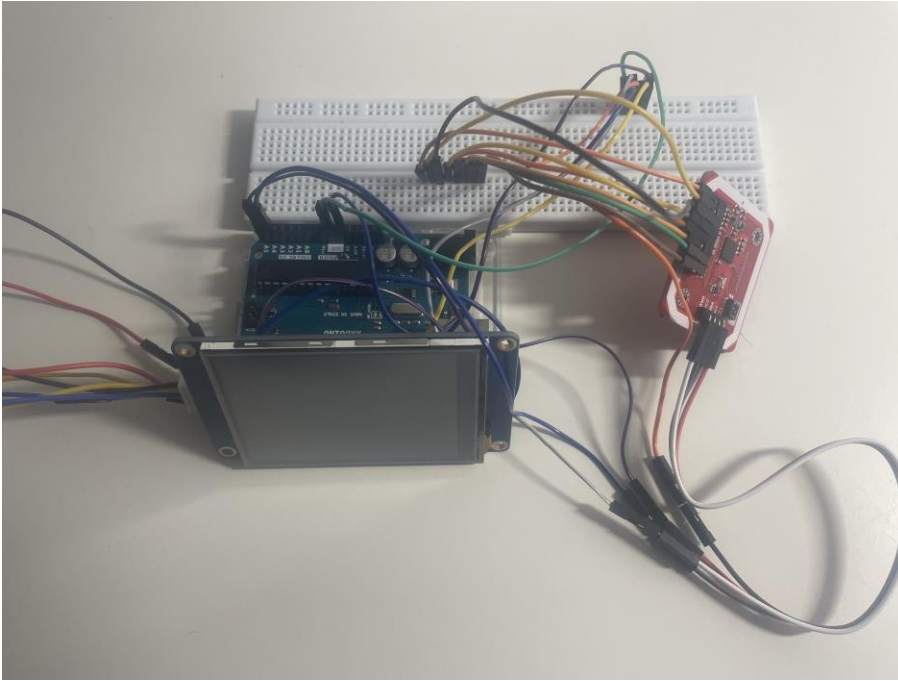
Partie 7 – Montage final et intégration

7.1 Description du montage

Le montage final du projet intègre l'ensemble des composants décrits précédemment en un système cohérent et fonctionnel. L'assemblage comprend les éléments suivants :

- ☐ Le module RFID PN532 connecté à l'Arduino UNO via le bus I2C, disposé sur une breadboard pour faciliter le câblage.
- ☐ L'écran Nextion connecté à l'Arduino via la liaison UART (broches TX/RX de l'Arduino, à 9600 bauds).
- ☐ Les capteurs de vitesse, tension et courant connectés aux entrées appropriées de l'Arduino.
- ☐ L'Arduino connecté au Raspberry Pi via USB pour la transmission des données.
- ☐ Le Raspberry Pi connecté au réseau Ethernet et au VPN Tailscale.

Le montage a été réalisé en deux phases : une phase de prototypage sur breadboard, permettant de valider les connexions et le code, suivie d'une phase d'intégration dans l'enclosure du banc d'essai.



7.2 Interface Nextion – conception

L'interface graphique de l'écran Nextion a été conçue avec le logiciel Nextion Editor. Elle comprend deux pages principales :

- ☐ Page d'accueil : affiche l'UID RFID de la voiture détectée, ainsi que deux boutons : 'Affichage' et 'Configuration'.
- ☐ Page d'affichage : présente les données de mesure en temps réel (vitesse en RPM, tension en V, courant en A) dans des champs texte dédiés.

La communication entre l'Arduino et l'écran Nextion se fait en envoyant des commandes ASCII via la liaison UART. Par exemple, pour mettre à jour la valeur affichée dans le champ t0 :

```
sendToNextion("t0.txt=\"" + uid + "\"");
```

Chaque commande est terminée par trois octets 0xFF (protocole Nextion).

7.3 Validation du montage

Des tests de validation ont été effectués sur le montage complet. Les résultats obtenus sont les suivants :

- ☐ Lecture RFID : la lecture de l'UID d'un tag Mifare Classic 1K est fonctionnelle et fiable. L'UID (ex : 4B C2 CD 01) s'affiche correctement sur le moniteur série Arduino et sur l'écran Nextion.
- ☐ Affichage Nextion : l'écran tactile affiche les données correctement. Les boutons 'Affichage' et 'Configuration' sont fonctionnels.
- ☐ Communication Arduino-Raspberry Pi : les données sont transmises via USB et reçues correctement par le Raspberry Pi.
- ☐ Enregistrement en base de données : les données transmises sont insérées dans la table 'performances' de MariaDB.

Partie 8 – Développement logiciel

8.1 Structure du projet logiciel

Le développement logiciel du projet repose sur plusieurs composants travaillant conjointement afin d'assurer le contrôle d'accès NFC et l'exploitation des données du banc d'essai.

L'architecture logicielle est composée des éléments suivants :

- **Firmware Arduino (C++)** : gestion de la lecture NFC via le module PN532, communication série avec le PC et affichage des informations sur l'écran Nextion.
- **Programme Python (UID_Check.py)** : assure la liaison entre l'Arduino et le serveur Raspberry Pi en transmettant les UID lus et en récupérant le résultat de l'authentification.
- **Application PHP (check_uid.php)** : vérifie les droits d'accès dans la base de données MariaDB et génère l'interface web de consultation.
- **Base de données MariaDB** : stockage des utilisateurs autorisés, des performances du banc d'essai et des journaux d'accès.
- **Interface Nextion (.HMI)** : affichage local des informations destinées à l'utilisateur.
- **Notebook Jupyter** : analyse et exploitation des données enregistrées.

Cette organisation modulaire facilite la maintenance du projet et permet de faire évoluer chaque composant indépendamment.

8.2 Firmware Arduino

Le firmware Arduino constitue le cœur du système de contrôle d'accès. Il assure la communication avec le lecteur NFC PN532 ainsi que l'affichage des résultats sur l'écran Nextion.

8.2.1 Initialisation du système

Lors de sa mise sous tension, l'Arduino exécute la fonction `setup()`.

Cette phase permet :

- l'initialisation de la liaison série USB à 115200 bauds ;
- l'initialisation de la liaison série logicielle vers le Nextion à 9600 bauds ;
- la réinitialisation de l'écran tactile grâce à la commande `rest` ;
- l'initialisation du module PN532 ;
- l'effacement des anciennes informations affichées ;
- l'affichage des messages de démarrage.

L'utilisateur est alors invité à présenter une carte NFC devant le lecteur.

8.2.2 Lecture et traitement des badges NFC

La fonction `loop()` surveille en permanence la présence d'un badge NFC.

Lorsqu'une carte est détectée :

1. l'UID est lu par le module PN532 ;
2. l'identifiant est formaté et converti en majuscules ;
3. l'information est envoyée au PC via la liaison série sous la forme :

UID:XXXXXXXX

4. l'Arduino attend ensuite la réponse du programme Python.

Un délai maximal de cinq secondes est prévu afin d'éviter tout blocage en cas de problème de communication.

8.2.3 Gestion de l'affichage Nextion

Après réception de la réponse du système d'authentification, l'Arduino met à jour l'écran Nextion.

Trois états sont possibles :

- **ACCÈS AUTORISÉ**
- **ACCÈS REFUSÉ**

• ERREUR DE COMMUNICATION

Lors des phases de développement, plusieurs difficultés ont été rencontrées concernant l'affichage dynamique de données complexes sur le Nextion. Afin de garantir la stabilité du système, une solution de contournement a été retenue : l'écran affiche uniquement le résultat de l'authentification tandis que les informations détaillées sont consultables via l'interface web.



8.3 Programme Python UID_Check.py

Le programme UID_Check.py est exécuté sur un ordinateur Windows connecté à l'Arduino. Son rôle est d'assurer la communication entre le système embarqué et le serveur hébergé sur le Raspberry Pi.

Le programme réalise les opérations suivantes :

- écoute du port COM5 à 115200 bauds ;
- réception des UID transmis par l'Arduino ;
- construction d'une requête HTTP vers le serveur PHP ;
- transmission de l'UID au Raspberry Pi ;
- réception de la réponse du serveur ;
- renvoi du résultat à l'Arduino.

L'utilisation de Python simplifie les échanges réseau et permet de séparer clairement les fonctions matérielles et applicatives.

8.4 Application PHP et contrôle d'accès

Le contrôle d'accès est assuré par le script PHP `check_uid.php` hébergé sur le Raspberry Pi. Ce script possède deux modes de fonctionnement.

Mode API

Lorsqu'il reçoit une requête contenant le paramètre : `?uid=XXXXXXXX&api=1`

le script :

- recherche l'UID dans la table utilisateurs ;
- vérifie si la carte est autorisée ;
- enregistre automatiquement l'événement dans la table `logs_acces` ;
- retourne la réponse OK ou KO.

Mode interface web

Le même script génère également une interface utilisateur consultable depuis un navigateur.

Cette page :

- utilise une photographie du banc d'essai comme arrière-plan ;
- affiche le résultat du dernier contrôle d'accès ;
- présente le nom et le prénom de l'utilisateur identifié ;
- affiche l'UID utilisé ;
- se rafraîchit automatiquement toutes les cinq secondes.

Cette interface constitue un outil de supervision simple et efficace pour les utilisateurs du banc d'essai.

8.5 Base de données MariaDB

La base de données centralise l'ensemble des informations du projet.

La table `utilisateurs` contient les cartes NFC autorisées ainsi que les informations associées aux utilisateurs.

La table `performances` stocke les données issues du banc d'essai afin de permettre leur exploitation ultérieure.

La table `logs_acces` enregistre chaque tentative d'authentification avec :

- l'UID détecté ;
- le nom de l'utilisateur ;
- le statut de l'accès ;
- la date et l'heure du contrôle.

Cette organisation permet d'assurer la traçabilité complète du système.

8.6 Notebook Jupyter – Analyse de données

Un notebook Jupyter a été développé afin d'exploiter les données enregistrées dans la base de données.

Cet outil permet :

- l'importation des données depuis MariaDB ou un fichier CSV ;
- la génération de graphiques de performance ;
- le calcul de statistiques descriptives ;
- la comparaison de plusieurs sessions d'essais.

Les résultats obtenus facilitent l'analyse des performances des véhicules et constituent une aide à l'optimisation des réglages du banc d'essai.

Partie 9 – Phase de consolidation avant l'oral

9.1 Objectifs de la phase de consolidation

La phase de consolidation vise à finaliser et intégrer les derniers éléments du projet avant l'oral.

Les travaux prévus dans cette phase concernent les domaines suivants :

- ☐ Création du PCB
- ☐ Mesure de vitesse avec le capteur LM393 et disque codeur : finalisation du code de calcul de la vitesse de rotation à partir du signal PWM du capteur LM393..
- ☐ Interface Nextion avancée : développement des pages d'affichage des graphiques et des historiques sur l'écran tactile.
- ☐ Intégration Grafana : configuration des tableaux de bord Grafana pour la visualisation des données de performance de la voiture RC en temps réel.

9.2 Création d'un PCB

Dans le cadre de la phase de consolidation du projet, une étude de conception d'un circuit imprimé (PCB) a été réalisée afin d'améliorer l'intégration des différents composants électroniques. L'objectif est de remplacer le câblage de prototypage par une solution plus fiable, plus compacte et plus robuste.

Ce PCB permettrait de regrouper les connexions entre l'Arduino, le lecteur NFC PN532, l'écran Nextion et les différents éléments d'alimentation. Cette évolution faciliterait également la maintenance du système et améliorerait sa qualité de réalisation.

Bien que la carte n'ait pas été fabriquée durant le projet, cette étude constitue une perspective d'amélioration pour une future version du banc d'essai.

9.3 Capteur LM393 – Mesure de vitesse

Le capteur LM393 associé à un disque codeur (encoder disk) à 20 encoches permet de mesurer la vitesse de rotation des rouleaux du banc d'essai, et par extension la vitesse des roues de la voiture RC. Le principe de fonctionnement est le suivant :

- ☐ Le disque codeur est fixé sur l'axe du rouleau, solidaire de la roue de la voiture testée.
- ☐ La fourche optique du capteur LM393 génère un signal numérique (impulsions) à chaque passage d'une encoche.
- ☐ L'Arduino comptabilise le nombre d'impulsions dans un intervalle de temps défini (100 ms), puis calcule la vitesse de rotation en tr/min.

La formule de calcul est : $N \text{ (tr/min)} = (\text{compteur_impulsions} / \text{nb_encoches}) \times (60\,000 / \text{délai_ms})$.
Avec nb_encoches = 20 et délai_ms = 100, on obtient : $N = \text{compteur} \times 30$.

Des tests préliminaires ont montré que le capteur peut mesurer des vitesses allant jusqu'à 30 000 tr/min avec une bonne précision, sous réserve que le disque codeur soit bien aligné avec la fourche optique.

9.4 Visualisation avec Grafana

Grafana est un outil open-source de visualisation et d'analyse de données. Intégré à notre serveur Raspberry Pi, il permet de créer des tableaux de bord interactifs à partir des données stockées dans MariaDB. Les tableaux de bord prévus sont :

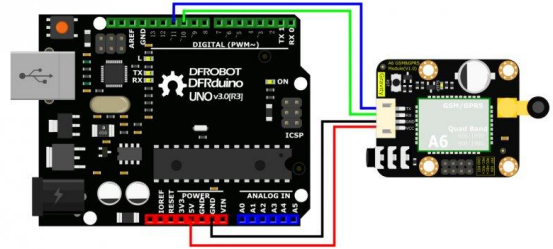
- ☐ Tableau de bord en temps réel : affichage de la vitesse, tension, courant et couple en cours de mesure.
- ☐ Historique par voiture : graphiques d'évolution des performances d'un véhicule donné sur plusieurs sessions.
- ☐ Comparaison multi-voitures : comparaison des performances de différents véhicules sur une même session.

Partie 10 – Étape proactive : module GSM A6

10.1 Présentation et motivation

En complément des fonctionnalités prévues dans le cahier des charges initial, nous avons identifié et proposé une fonctionnalité supplémentaire à valeur ajoutée : l'ajout d'un module de communication GSM (Global System for Mobile Communications) permettant l'envoi et la réception de SMS.

Cette fonctionnalité proactive vise à permettre la notification des résultats de performance directement sur le téléphone du pilote, sans nécessiter l'installation d'une application ou l'accès à une interface web. Elle est particulièrement pertinente dans les situations où la connectivité réseau est limitée ou absente sur le lieu de la compétition.



10.2 Module A6 – Caractéristiques techniques

Le module A6 est un module GSM/GPRS développé par AI-Thinker. Il permet l'envoi et la réception de SMS, ainsi que des communications vocales et des connexions GPRS. Ses caractéristiques principales sont :

- ☐ Fréquences supportées : réseau quad-band 850 / 900 / 1800 / 1900 MHz. Compatible avec la majorité des réseaux GSM mondiaux.
- ☐ Tension d'alimentation : 4,8 à 9V DC. Courant maximal : 2A (en émission).
- ☐ Interface de communication : port série TTL ou RS232, à 115200 bps.
- ☐ Protocole de commande : commandes AT standard (Hayes), bien documentées et compatibles avec la plupart des bibliothèques Arduino.
- ☐ Format : module compact, facilement intégrable dans un système embarqué.

10.3 Câblage et intégration

Le module A6 est connecté à l'Arduino via une liaison UART (TTL, 115200 bps). Étant donné que l'Arduino UNO ne dispose que d'une liaison UART matérielle (partagée avec USB et Nextion), nous utilisons une bibliothèque de liaison série logicielle (SoftwareSerial) pour la communication avec le module A6.

Le câblage est le suivant :

- ☐ VCC du module A6 → alimentation externe 5V (minimum 2A recommandés pour les pics de courant en émission GSM).
- ☐ GND du module A6 → GND de l'Arduino et de l'alimentation.
- ☐ TX du module A6 → broche de réception de SoftwareSerial (ex : pin 10 Arduino).

- ❑ RX du module A6 → broche d'émission de SoftwareSerial (ex : pin 11 Arduino).

10.4 Fonctionnalités prévues

Les fonctionnalités envisagées avec le module GSM A6 sont :

- ❑ Envoi automatique d'un SMS au pilote à la fin de chaque session de test, avec un résumé des performances (vitesse max, tension min, courant max, couple).
- ❑ Notification d'alerte par SMS en cas de dépassement des seuils de performance définis par les normes FVRC (ex : vitesse maximale autorisée).
- ❑ Commande à distance par SMS : possibilité de déclencher une session de mesure ou d'interroger l'état du système par l'envoi d'un SMS au module.

L'implémentation de ces fonctionnalités nécessite l'utilisation des commandes AT standard : AT+CMGF=1 pour le mode SMS texte, AT+CMGS pour l'envoi d'un SMS, et la gestion des interruptions pour la réception.

Partie 11 – Difficultés rencontrées et solutions apportées

11.1 Difficultés techniques

11.1.1 Choix du protocole pour le module PN532

La première difficulté rencontrée a été le choix du protocole de communication pour le module PN532. Initialement, nous avons tenté d'utiliser le protocole SPI, mais des conflits avec les autres périphériques SPI sur le bus Arduino ont causé des instabilités. Nous avons finalement opté pour le protocole I2C, qui s'est avéré plus stable et plus simple à mettre en œuvre dans notre contexte.

11.1.2 Gestion des liaisons UART multiples

L'Arduino UNO ne dispose que d'une seule interface UART matérielle, partagée entre la communication USB (avec le Raspberry Pi) et l'affichage sur l'écran Nextion. Nous avons résolu ce problème en utilisant la bibliothèque SoftwareSerial pour créer une liaison UART logicielle sur deux broches numériques dédiées à l'écran Nextion, libérant ainsi la liaison UART matérielle pour la communication avec le Raspberry Pi.

11.1.3 Configuration du Nextion Editor

La prise en main du logiciel Nextion Editor a demandé un temps d'apprentissage significatif. La gestion des événements tactiles, la mise à jour dynamique des champs texte et la communication avec l'Arduino via les commandes ASCII ont nécessité plusieurs itérations avant d'obtenir un comportement stable et fiable.

Aussi, lors du développement du projet, des difficultés ont été rencontrées concernant la communication avec l'écran Nextion.

L'objectif initial consistait à afficher dynamiquement les informations complètes de l'utilisateur identifié directement sur l'écran tactile. Cependant, plusieurs limitations logicielles ont été observées lors des essais, notamment lors des mises à jour dynamiques des différents champs de texte.

Afin de garantir la stabilité du système, une solution de contournement a été adoptée. L'écran Nextion a été limité à l'affichage des informations essentielles : accès autorisé, accès refusé ou erreur de communication.

Les informations détaillées telles que l'UID de la carte, le nom de l'utilisateur et l'historique des accès sont désormais affichées sur l'interface web hébergée sur le Raspberry Pi. Cette solution améliore également la lisibilité des informations et facilite leur consultation.

11.1.4 Configuration du VPN Tailscale

La configuration de Tailscale sur le Raspberry Pi a été facilitée par la bonne documentation officielle. Cependant, des problèmes ont été rencontrés lors de la première connexion, liés à la version du kernel Linux et au pare-feu. Ces problèmes ont été résolus en mettant à jour le système et en configurant les règles iptables appropriées.

11.2 Difficultés organisationnelles

Outre les difficultés techniques, nous avons rencontré quelques difficultés organisationnelles :

- ☐ Coordination entre les deux membres du binôme : la répartition des tâches et la synchronisation des développements ont nécessité une communication régulière et l'utilisation d'un outil de gestion de version (Git).
- ☐ Disponibilité du matériel : certains composants (notamment le module A6 GSM et les capteurs de courant) ont dû être commandés avec un délai, ce qui a légèrement retardé certaines étapes du projet.
- ☐ Accès au banc d'essai : le banc d'essai appartenant au club partenaire FVRC, sa disponibilité pour les tests n'était pas toujours garantie. Certains tests ont donc dû être effectués avec du matériel de substitution.

Partie 12 – Planning et état d'avancement

12.2 Bilan intermédiaire

À la date de la remise du rapport de projet, les travaux suivants ont été accomplis avec succès :

- ☐ Identification et acquisition du matériel nécessaire.
- ☐ Prototypage et validation du module RFID/NFC avec protocole I2C.
- ☐ Intégration de l'écran Nextion avec affichage de l'UID RFID et navigation par boutons.
- ☐ Installation et configuration complète du serveur Apache, PHP, MariaDB et phpMyAdmin sur Raspberry Pi.
- ☐ Mise en place du réseau VPN Tailscale avec 3 machines connectées.
- ☐ Début de développement de l'interface web PHP.

Les travaux en cours ou à initier sont :

- ☐ Réalisation du PCB
- ☐ Finalisation du code de mesure de vitesse avec le capteur LM393.
- ☐ Mise en place du module GSM A6 (fonctionnalité proactive).
- ☐ Tests et validation globale du système intégré.

Partie 13 – Perspectives et travaux futurs

13.1 Travaux restants avant l'oral

Les travaux qui restent à accomplir avant la l'oral sont décrits ci-dessous, par ordre de priorité :

13.1.1 PCB (priorité haute)

13.1.2 Finalisation du code de mesure (priorité haute)

La mesure de vitesse avec le capteur LM393 est partiellement implémentée. Il convient de finaliser le code de comptage des impulsions avec des interruptions matérielles (`attachInterrupt()`) pour garantir la fiabilité des mesures à haute vitesse. De même, les capteurs de tension et de courant doivent être calibrés et leurs codes de lecture validés sur le banc d'essai réel.

13.1.3 Intégration Grafana (priorité moyenne)

Les tableaux de bord Grafana doivent être configurés pour afficher les données de performance en temps réel depuis la base de données MariaDB. La configuration du plugin MariaDB pour Grafana et la création des requêtes SQL associées restent à effectuer.

13.1.4 Module GSM A6 (priorité basse)

L'intégration du module GSM A6 est une fonctionnalité proactive. Elle sera mise en œuvre si le temps disponible le permet, après l'achèvement des fonctionnalités prioritaires.

13.2 Améliorations potentielles à long terme

Au-delà de la revue finale, plusieurs améliorations pourraient être envisagées pour enrichir le système :

- ☐ Intégration d'un écran de plus grande taille (7 pouces) pour une meilleure lisibilité des données en situation de compétition.
- ☐ Ajout d'un module WiFi (ESP8266 ou ESP32) pour une transmission sans fil des données entre l'Arduino et le Raspberry Pi, évitant le câble USB.
- ☐ Développement d'une application mobile dédiée (Android/iOS) pour la consultation des résultats depuis un smartphone, avec notifications push.
- ☐ Intégration d'un système de chronométrage complémentaire, permettant de mesurer la durée des sessions de test et de calculer l'autonomie de la batterie.
- ☐ Mise en place d'une interface de saisie des réglages mécanique (débattement, jeu différentiel, etc.) pour associer les réglages aux mesures de performance.

Conclusion

Ce projet d'instrumentation d'un banc d'essai pour voitures RC constitue un défi technique ambitieux et formateur, qui mobilise l'ensemble des compétences acquises au cours de notre formation en BTS CIEL ER. Il nous a permis de mettre en pratique nos connaissances en électronique embarquée, en programmation, en réseaux et en systèmes d'information, dans un contexte applicatif concret et stimulant.

À la date de la revue de projet intermédiaire, des progrès significatifs ont été réalisés. Les composants clés du système – le module NFC/RFID, l'écran Nextion, le serveur Raspberry Pi avec sa base de données MariaDB et son VPN Tailscale – ont été implémentés et validés avec succès. Les premiers résultats obtenus sont conformes aux attentes, avec une lecture RFID fiable, un affichage fonctionnel et une infrastructure serveur opérationnelle.

Cependant, des travaux importants restent à accomplir avant la revue finale : finalisation du code de mesure des capteurs physiques (vitesse, tension, courant), développement complet de l'interface web avec Grafana, et intégration de l'ensemble en un système cohérent. Ces travaux sont planifiés et font l'objet d'un suivi rigoureux.

La démarche proactive adoptée pour l'ajout du module GSM A6 témoigne de notre engagement à dépasser les exigences minimales du projet et à apporter une valeur ajoutée réelle aux utilisateurs finaux – les pilotes et les techniciens du club FVRC Mini RC de Marigny.

Ce projet nous a également appris la gestion de projet, la résolution de problèmes techniques complexes, et l'importance de la communication et de la collaboration au sein d'un binôme. Ces compétences transversales seront précieuses dans notre future vie professionnelle.

Nous sommes confiants dans notre capacité à mener ce projet à son terme et à présenter un système d'instrumentation pleinement fonctionnel lors de la revue finale.

Bibliographie et références

Documentation technique

- NXP Semiconductors. PN532/C1 – User Manual. Rev. 3.3. NXP Semiconductors, 2014. [En ligne]. Disponible sur : <https://www.nxp.com/docs/en/user-guide/141520.pdf>
- Adafruit Industries. Adafruit-PN532 Library Documentation. GitHub, 2024. [En ligne]. Disponible sur : <https://github.com/adafruit/Adafruit-PN532>
- Nextion. Nextion HMI Solution – NX3224T028 Datasheet. ITEAD Studio. [En ligne]. Disponible sur : <https://nextion.tech/datasheets/>
- Raspberry Pi Foundation. Raspberry Pi 3 Model B+ Product Brief. 2018. [En ligne]. Disponible sur : <https://www.raspberrypi.com>
- MariaDB Corporation. MariaDB Documentation. 2024. [En ligne]. Disponible sur : <https://mariadb.com/kb/en/documentation/>
- Apache Software Foundation. Apache HTTP Server Documentation. Version 2.4. 2024. [En ligne]. Disponible sur : <https://httpd.apache.org/docs/2.4/>

Références logicielles

- Tailscale Inc. Tailscale Documentation. 2024. [En ligne]. Disponible sur : <https://tailscale.com/kb/>
- Grafana Labs. Grafana Documentation. 2024. [En ligne]. Disponible sur : <https://grafana.com/docs/>
- phpMyAdmin Contributors. phpMyAdmin Documentation. Version 5.2. 2024. [En ligne]. Disponible sur : <https://www.phpmyadmin.net/docs/>
- Project Jupyter. Jupyter Notebook Documentation. 2024. [En ligne]. Disponible sur : <https://jupyter-notebook.readthedocs.io/>

Normes et réglementations

- Fédération de Voiture Radio-Commandée (FVRC). Règlement technique des voitures radio-commandées de compétition. Édition 2025-2026.
- IEC 61000-4-2. Electromagnetic compatibility (EMC) – Part 4-2: Testing and measurement techniques – Electrostatic discharge immunity test.

Annexes

Annexe A – Schéma de câblage complet

Le schéma de câblage complet du système d'instrumentation présente les connexions entre l'Arduino UNO, le module PN532 (I2C), l'écran Nextion (UART), les capteurs de vitesse (LM393), de tension et de courant, ainsi que la liaison USB avec le Raspberry Pi. Ce schéma constitue le document de référence pour l'assemblage physique du système.

Connexions principales résumées :

- ☐ PN532 SDA → Arduino A4 | PN532 SCL → Arduino A5 | PN532 VCC → 5V | PN532 GND → GND
- ☐ Nextion TX → Arduino D10 (SoftSerial RX) | Nextion RX → Arduino D11 (SoftSerial TX)
- ☐ LM393 OUT → Arduino D2 (interruption INT0) | LM393 VCC → 5V | LM393 GND → GND
- ☐ Sonde tension → Arduino A0 | Sonde courant → Arduino A1
- ☐ Arduino USB → Raspberry Pi USB (communication série à 115200 bds)

Annexe B – Commandes AT du module GSM A6

Les principales commandes AT utilisées pour le module GSM A6 sont :

- ☐ AT – Test de communication (réponse attendue : OK)
- ☐ AT+CSQ – Vérification de la qualité du signal (RSSI)
- ☐ AT+CMGF=1 – Sélection du mode SMS texte
- ☐ AT+CMGS="+33XXXXXXXXXX" – Envoi d'un SMS au numéro spécifié
- ☐ AT+CNMI=1,2,0,0,0 – Configuration de la notification de réception de SMS
- ☐ AT+CMGL="ALL" – Liste de tous les SMS reçus